

```

"""
FastAPI service for Patient Risk Scoring and Claim Outcome prediction.

Models
-----
patient_risk_model    : predicts risk_score → Low | Medium | High    (v1.0)
claim_outcome_model   : predicts claim_status → Paid | Pending | Rejected (v1.0)

Endpoints
-----
GET   /health           - overall service liveness
GET   /health/patient-risk - patient risk model readiness
GET   /health/claim-outcome - claim outcome model readiness
POST  /predict/patient-risk - patient risk prediction
POST  /predict/claim-outcome - claim outcome prediction

Logging
-----
Every prediction is logged to stdout and predictions.log with:
    prediction_id, model name, version, UTC timestamp, feature hash (SHA-256), label, confidence
"""

import hashlib
import json
import logging
import uuid
from contextlib import asynccontextmanager
from datetime import datetime, timezone
from pathlib import Path
from typing import Any, Literal

import sys

import pandas as pd
from fastapi import FastAPI, HTTPException
from fastapi.responses import FileResponse
from pydantic import BaseModel, Field

# Validation module lives in Phase 6
_PHASE6_DIR = (
    Path(__file__).resolve().parent.parent
    / "Phase 6 - Monitoring, Drift Detection, and Governance"
)
if str(_PHASE6_DIR) not in sys.path:
    sys.path.insert(0, str(_PHASE6_DIR))
from validate import validate_payload # noqa: E402

# -----
# Logging setup
# -----
_LOG_PATH = Path(__file__).resolve().parent / "logs" / "predictions.log"
_LOG_PATH.parent.mkdir(parents=True, exist_ok=True)
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)-8s %(message)s",
    handlers=[
        logging.StreamHandler(),

```

```

        logging.FileHandler(_LOG_PATH),
    ],
)
logger = logging.getLogger(__name__)

# -----
# Model paths and versions
# -----
_MODEL_DIR = Path(__file__).resolve().parent.parent.parent / "Model Files"

_PATIENT_RISK_PATH = _MODEL_DIR / "patient_risk_model.pkl"
_CLAIM_OUTCOME_PATH = _MODEL_DIR / "claim_outcome_model.pkl"

# Feature schemas (for validation)
_PATIENT_RISK_SCHEMA_PATH = _MODEL_DIR / "patient_risk_model_feature_schema.json"
_CLAIM_OUTCOME_SCHEMA_PATH = _MODEL_DIR / "claim_outcome_model_feature_schema.json"

PATIENT_RISK_VERSION = "1.0"
CLAIM_OUTCOME_VERSION = "1.0"

_RAW_DATA_DIR = Path(__file__).resolve().parent.parent.parent / "Raw Data"

# -----
# Internal helpers
# -----
_models: dict[str, Any] = {}
_metadata: dict[str, list] = {}
_schemas: dict[str, dict] = {}

def _load_metadata() -> dict[str, list]:
    """Read distinct categorical values for claim-outcome fields from the raw CSVs."""
    patients = pd.read_csv(_RAW_DATA_DIR / "patients.csv")
    visits = pd.read_csv(_RAW_DATA_DIR / "visits.csv")
    return {
        "city": sorted(patients["city"].dropna().unique().tolist()),
        "insurance_provider": sorted(patients["insurance_provider"].dropna().unique().tolist()),
        "department": sorted(visits["department"].dropna().unique().tolist()),
        "visit_type": sorted(visits["visit_type"].dropna().unique().tolist()),
    }

def _load_model(path: Path) -> Any:
    from pycaret.classification import load_model # type: ignore
    # load_model expects the path WITHOUT the .pkl extension
    return load_model(str(path.with_suffix("")))

def _feature_hash(features: dict) -> str:
    """SHA-256 of the canonically serialised feature dict."""
    payload = json.dumps(features, sort_keys=True, default=str)
    return hashlib.sha256(payload.encode()).hexdigest()

def _run_pycaret_predict(model: Any, df: pd.DataFrame) -> tuple[str, float]:
    """
    Run inference through a PyCaret classification pipeline using the
    sklearn interface directly – no PyCaret setup() session required.
    Returns (predicted_label, prediction_score).
    """

```

```

"""
try:
    label = str(model.predict(df)[0])
    proba = model.predict_proba(df)[0]
    score = float(proba.max())
    return label, score
except Exception as exc:
    raise HTTPException(status_code=500, detail=f"Prediction failed: {exc}") from exc

def _log_prediction(
    *,
    model_name: str,
    model_version: str,
    prediction_id: str,
    timestamp: str,
    feature_hash: str,
    label: str,
    score: float,
) -> None:
    logger.info(
        "PREDICTION | id=%s | model=%s v%s | ts=%s | hash=%s | label=%s | score=%.4f",
        prediction_id,
        model_name,
        model_version,
        timestamp,
        feature_hash,
        label,
        score,
    )

# -----
# Lifespan – load both models once at startup
# -----
@asynccontextmanager
async def lifespan(app: FastAPI): # noqa: ARG001
    logger.info("Loading patient_risk_model from %s", _PATIENT_RISK_PATH)
    _models["patient_risk"] = _load_model(_PATIENT_RISK_PATH)

    logger.info("Loading claim_outcome_model from %s", _CLAIM_OUTCOME_PATH)
    _models["claim_outcome"] = _load_model(_CLAIM_OUTCOME_PATH)

    logger.info("Loading feature schemas for validation...")
    _schemas["patient_risk"] = json.loads(_PATIENT_RISK_SCHEMA_PATH.read_text())
    _schemas["claim_outcome"] = json.loads(_CLAIM_OUTCOME_SCHEMA_PATH.read_text())

    logger.info("Loading categorical metadata from raw CSVs...")
    _metadata.update(_load_metadata())
    logger.info("Metadata loaded: %s", {k: len(v) for k, v in _metadata.items()})

    logger.info("Both models loaded and ready.")
    yield

    _models.clear()
    _metadata.clear()
    logger.info("Models unloaded.")

```

```

# -----
# Application
# -----
app = FastAPI(
    title="Healthcare ML Prediction API",
    version="1.0.0",
    description=(
        "Predict **patient visit risk** (Low / Medium / High) "
        "and **insurance claim outcomes** (Paid / Pending / Rejected).",
    ),
    lifespan=lifespan,
)

# -----
# Request / Response schemas
# -----
class PatientRiskRequest(BaseModel):
    chronic_flag: int = Field(
        ..., ge=0, le=1, description="1 if the patient has a chronic condition, else 0"
    )
    gender: Literal["M", "F"] = Field(..., description="Patient gender")
    visit_frequency: int = Field(..., ge=1, description="Total number of visits by this patient")

class ClaimOutcomeRequest(BaseModel):
    age: int = Field(..., description="Patient age in years")
    gender: Literal["M", "F"] = Field(..., description="Patient gender")
    city: str = Field(..., min_length=1, description="Patient city")
    insurance_provider: str = Field(..., min_length=1, description="Insurance provider name")
    chronic_flag: int = Field(
        ..., ge=0, le=1, description="1 if the patient has a chronic condition, else 0"
    )
    department: str = Field(..., min_length=1, description="Hospital department")
    visit_type: str = Field(..., min_length=1, description="Type of visit (e.g. Emergency, Routine)")
    length_of_stay_hours: float = Field(..., ge=0, description="Length of stay in hours")
    billed_amount: float = Field(..., ge=0, description="Total amount billed")
    payment_days: float = Field(..., ge=0, description="Days from billing to payment")
    visit_frequency: int = Field(..., ge=1, description="Total number of visits by this patient")
    lag_time: int = Field(..., description="Days between visit date and billing date")

class PredictionResponse(BaseModel):
    prediction_id: str = Field(..., description="Unique ID for this prediction")
    timestamp: str = Field(..., description="UTC timestamp of prediction (ISO 8601)")
    model_version: str = Field(..., description="Model version used")
    prediction: str = Field(..., description="Predicted label")
    confidence: float = Field(..., description="Model confidence for the predicted label")
    feature_hash: str = Field(..., description="SHA-256 hash of the input features")

class HealthResponse(BaseModel):
    status: str
    model: str | None = None
    version: str | None = None
    models_loaded: list[str] | None = None
    timestamp: str

```

```

# -----
# UI
# -----
@app.get("/", include_in_schema=False)
async def ui():
    return FileResponse(Path(__file__).parent / "index.html")

@app.get("/metadata/claim-outcome", tags=["Metadata"])
async def claim_outcome_metadata():
    """Return valid categorical values for the claim outcome prediction form."""
    return _metadata

# -----
# Health endpoints
# -----
@app.get("/health", response_model=HealthResponse, tags=["Health"])
async def health():
    """Overall service liveness – returns loaded model names."""
    return HealthResponse(
        status="ok",
        models_loaded=list(_models.keys()),
        timestamp=datetime.now(timezone.utc).isoformat(),
    )

@app.get("/health/patient-risk", response_model=HealthResponse, tags=["Health"])
async def health_patient_risk():
    """Patient risk model readiness check."""
    if "patient_risk" not in _models:
        raise HTTPException(status_code=503, detail="Patient risk model not loaded")
    return HealthResponse(
        status="ok",
        model="patient_risk_model",
        version=PATIENT_RISK_VERSION,
        timestamp=datetime.now(timezone.utc).isoformat(),
    )

@app.get("/health/claim-outcome", response_model=HealthResponse, tags=["Health"])
async def health_claim_outcome():
    """Claim outcome model readiness check."""
    if "claim_outcome" not in _models:
        raise HTTPException(status_code=503, detail="Claim outcome model not loaded")
    return HealthResponse(
        status="ok",
        model="claim_outcome_model",
        version=CLAIM_OUTCOME_VERSION,
        timestamp=datetime.now(timezone.utc).isoformat(),
    )

# -----
# Prediction endpoints
# -----
@app.post(
    "/predict/patient-risk",
    response_model=PredictionResponse,

```

```

tags=["Predictions"],
summary="Predict patient visit risk",
)
async def predict_patient_risk(request: PatientRiskRequest):
    """
    Predict the risk score for a patient visit.

    **Returns:** `Low` | `Medium` | `High`
    """
    if "patient_risk" not in _models:
        raise HTTPException(status_code=503, detail="Patient risk model not loaded")

    features = request.model_dump()

    is_valid, validation_errors = validate_payload(features, _schemas["patient_risk"])
    if not is_valid:
        raise HTTPException(status_code=422, detail={"validation_errors": validation_errors})

    df = pd.DataFrame([features])
    df["chronic_flag"] = df["chronic_flag"].astype("int8")
    df["visit_frequency"] = df["visit_frequency"].astype("int8")

    label, score = _run_pycaret_predict(_models["patient_risk"], df)

    prediction_id = str(uuid.uuid4())
    timestamp = datetime.now(timezone.utc).isoformat()
    feature_hash = _feature_hash(features)

    _log_prediction(
        model_name="patient_risk_model",
        model_version=PATIENT_RISK_VERSION,
        prediction_id=prediction_id,
        timestamp=timestamp,
        feature_hash=feature_hash,
        label=label,
        score=score,
    )

    return PredictionResponse(
        prediction_id=prediction_id,
        timestamp=timestamp,
        model_version=PATIENT_RISK_VERSION,
        prediction=label,
        confidence=score,
        feature_hash=feature_hash,
    )

@app.post(
    "/predict/claim-outcome",
    response_model=PredictionResponse,
    tags=["Predictions"],
    summary="Predict insurance claim outcome",
)
async def predict_claim_outcome(request: ClaimOutcomeRequest):
    """
    Predict the outcome of an insurance claim.

    **Returns:** `Paid` | `Pending` | `Rejected`

```



```

"""
if "claim_outcome" not in _models:
    raise HTTPException(status_code=503, detail="Claim outcome model not loaded")

features = request.model_dump()

is_valid, validation_errors = validate_payload(features, _schemas["claim_outcome"])
if not is_valid:
    raise HTTPException(status_code=422, detail={"validation_errors": validation_errors})

df = pd.DataFrame([features])
df["age"] = df["age"].astype("int8")
df["chronic_flag"] = df["chronic_flag"].astype("int8")
df["visit_frequency"] = df["visit_frequency"].astype("int8")
df["length_of_stay_hours"] = df["length_of_stay_hours"].astype("float32")
df["billed_amount"] = df["billed_amount"].astype("float32")
df["payment_days"] = df["payment_days"].astype("float32")
df["lag_time"] = df["lag_time"].astype("int32")

label, score = _run_pycaret_predict(_models["claim_outcome"], df)

prediction_id = str(uuid.uuid4())
timestamp = datetime.now(timezone.utc).isoformat()
feature_hash = _feature_hash(features)

_log_prediction(
    model_name="claim_outcome_model",
    model_version=CLAIM_OUTCOME_VERSION,
    prediction_id=prediction_id,
    timestamp=timestamp,
    feature_hash=feature_hash,
    label=label,
    score=score,
)

return PredictionResponse(
    prediction_id=prediction_id,
    timestamp=timestamp,
    model_version=CLAIM_OUTCOME_VERSION,
    prediction=label,
    confidence=score,
    feature_hash=feature_hash,
)

# -----
# Entry point – supports hot-reload when run directly.
# From the Capstone root you can also use:
#   uvicorn Api:app --reload --app-dir "Notebooks/Phase 5 – Deployment and API Integration"
# -----
if __name__ == "__main__":
    import uvicorn

    uvicorn.run(
        "Api:app",
        host="127.0.0.1",
        port=8000,
        reload=True,
    )

```

```
) app_dir=str(Path(__file__).parent),
```